프로젝트 지식과 CLI 운영

파일 버전과 근거 및 변경 영향을 관리하는 실행 도구

2026년 10월 23일 · 최종 통합 제안 버전 2.0

Aethon과 별도인 신규 사업을 위한 완성형 요구서다. 사업·성능 수치는 검증할 목표이며 현재 달성 실적이 아니다. 기능과 API·스키마는 설계 계약이며 실제 플랫폼 테스트는 실행하지 않았다. 별도 제공하는 문서 관리 CLI와 원가 계산기는 로컬 검증 범위를 구분해 기록한다.

이 문서에서 확인할 내용

- 제품 속 아이돌 지식과 프로젝트 문서 지식의 분리
- Source·Claim·Requirement·API·Test의 연결
- 현재 기준선·과거 버전·재검토 기한
- 파일과 객체를 등록하고 승인하는 CLI
- 사업·제품·PPT 작업용 문맥 생성
- 가격·요구사항 변경의 영향 추적
- Git·CLI 에이전트 협업과 충돌 복구
- 실제 로컬 시험 결과와 제품 미검증 범위

기능 FR · 화면 SCR · 흐름 UFL · 지표 MET · 실험 EXP · 수용 시험 AT · 통합 시험 IT · 기술 TCH · 비기능 NFR

프로젝트 지식 온톨로지와 CLI 운영 요구서

목적과 이번에 구현한 범위

새 프로젝트에서 사업계획, 제품 요구서, 기술 계약, 화면, 실험, 발표자료를 계속 만들면 같은 숫자와 요구가 여러 파일에 반복된다. 이때 파일명에 최종, final2, 날짜를 붙이는 것만으로는 무엇이 현재 기준인지, 누가 어떤 범위를 승인했는지, 어느 PPT가 오래된 원가를 쓰는지 판단하기 어렵다.

이를 위해 **문서 파일과 주장·결정·요구·검증을 안정 ID로 연결하는 프로젝트 지식 계층**을 둔다. 최신성은 승인 manifest가 결정하고, 파일 바이트가 바뀌면 해시로 감지하며, 출처 또는 가정이 바뀌면 그것을 사용한 사업계획·요구사항·PPT를 영향 목록에 올린다. 이 계층의 원본은 Markdown/JSON과 보존한 파일이다. Git은 변경 이력과 복원에 사용한다. 초기 구현에는 DB·벡터 검색·그래프 서버가 필요하지 않다.

이번 산출물에는 제안 문서와 함께 **실행 가능한 Python 표준라이브러리 CLI**를 제공했다. project_kit/kb.py는 검증, 검색, 변경 영향 추적, 업무별 문맥 생성, 기존 카탈로그 import, 명시적 등록·승격·보관을 실행한다. 플랫폼 영상 생성·고객 IP 기억·결제·웹 게시를 실행하지 않는다. 실제 통과한 시험은 이 프로젝트 CLI의 fixture 시험이며, 제품 품질·고객 수요·제작 원가의 실측 증거와 구분한다.

관련 요구는 output/knowledge_management_requirements.json의 **KMR-001~020**에 따로 정리했다. 기존 제품 FR-001~096의 수에 문서 관리 기능을 섞지 않는다. 사업 요구는 output/business_requirements.json의 BUS-001~015, 외주 비교·원가·작업 계측의 추가 실험은 output/comparison_protocol.json의 EXP-C01~04에 있다. 이 세 파일도 실제 CLI importer가 읽고 객체로 등록한다.

두 온톨로지의 경계

구분	제품 런타임 온톨로지	프로젝트 지식 온톨로지
대상	고객 아이돌·그룹·캐릭터·활동기·곡·장면	출처·주장·가정·결정·요구·화면·API·시험·문서
핵심 질문	이번 장면에 어떤 공식 설정과 선호를 적용하는가	지금 어떤 버전과 근거로 작업해야 하는가
승인	공식 설정·제작·배포·권리 승인을 분리	문서 검토·요구 채택·검증 수준을 분리
저장	앞 장의 제품 PostgreSQL·자산·스냅샷 설계	JSON sidecar·Markdown·파일 해시·Git
결과	지속 기억을 참조하는 음악영상 제작	현재 근거로 사업·제품·발표 작업을 이어가는 문맥

두 계층은 설계 원칙을 공유하지만 데이터를 섞지 않는다. 프로젝트 문서가 “얼굴 유지”를 요구한다고 고객의 얼굴 자산이나 정사가 프로젝트 registry에 들어가는 것은 아니다. 이 키트에 있는 FR·API·실험 기록은 제품에 대한 설계 지식이며, 실제 고객 데이터 저장소가 아니다. Aethon의 자산·고객·검증·프로젝트 상태도 승계하지 않는다.

원본과 파생물의 구성

경로	역할	변경 원칙
sources/current_spec/	현재 전달 문서의 동결 사본	승인된 파일 바이트 보존
archive/v1/	이전 문서와 근거	기본 문맥 제외, 자동 삭제 금지
docs/	새 초안·가정·결정·문서 버전	작업별 새 파일과 명시적 버전
registry/knowledge.json	객체·관계·head·감사 원본	CLI 잠금·revision 검사 후 변경
generated/context/	업무별 제한 문맥과 receipt	언제든 원본에서 재생성 가능
tests/	CLI fixture와 실행 기록	제품 실증으로 승격하지 않음

현재 폴더 이름은 편의적인 분류다. 실제 최신 기준은 heads={canonical_key: record_id}가 결정한다. 같은 canonical key에 새 문서를 등록해도 draft인 동안은 기존 승인 버전이 기준이다. 사용자가 새 버전을 검토한 뒤 승격하면 기존 버전은 superseded가 되고 새 head가 선택된다. 파일 수정 시각과 final이라는 단어는 선택 기준으로 사용하지 않는다.

원문에는 가능한 한 사람이 검토할 수 있는 Markdown/JSON을 남긴다. DOCX/PDF/PPTX는 해당 텍스트 원문·사용한 근거와 연결하고 파일 해시를 보존한다. CLI는 바이너리의 해시를 확인하지만 그 안의 모든 문장을 추출하거나 PPT를 렌더하지 않는다.

객체 안정 ID 버전과 관계

등록 종류는 Source, Claim, Assumption, Decision, Requirement, Screen, API, Test, Metric, Experiment, Document의 11개다. 각 객체에는 버전에 고유한 id, 버전을 관통하는 canonical_key, 명시적 version, 상태, 유효 시점, 검토 기한, 원문 위치와 관계가 있다.

예를 들어 Requirement:FR-001은 안정적인 논리 키다. SPEC1:FR-001, SPEC2:FR-001은 각 기준선의 구체적 레코드다. FR-001이라는 원래 별칭도 보존한다. 같은 별칭의 버전이 여러 개면 현재 head를 선택하고, 현재 기준이 없어 모호하면 정확한 ID를 요구한다. 새 버전은 새 ID로 만들며 이전 ID의 의미를 조용히 고치지 않는다.

Source의 citation_id는 실제 출처 식별자다. Claim ID는 그 출처를 보고 작성한 주장의 식별자다. 출처가 있다고 그 출처에서 추론한 모든 결론이 검증된 것은 아니다. Assumption은 모델링에 사용한 전제이고, Metric은 분모·목표·관측값의 정의이며, Experiment는 가정을 검증할 절차와 실행 결과다.

저장 관계	의미	자동 영향 전파
Claim → supported_by → Source	주장의 근거	수행
Decision → depends_on → Claim	결정의 전제	수행
API/Screen → implements → Requirement	설계상 구현 연계	수행; 구현 완료 의미 없음
Test → verifies → Requirement/API	시험 정의의 대상	수행; 통과 의미 없음
Metric/Experiment → measures/tests → 대상	계측·실험 연계	수행
Document → derived_from/documents → 근거	사업계획·PPT의 사용 근거	수행
객체 → references → 객체	단순 관련 항목	영향 전파에서 제외

저장 방향은 소비자에서 근거를 향한다. impact는 반대 방향으로 탐색하여 변경이 영향을 미칠 소비자를 반환한다. Source 변경에서 Claim, 사업 결정, 사업계획, PPT로 이어지는 경로가 목록에 나타난다. 선언하지 않은 관계는 추적할 수 없으므로, 중요한 슬라이드의 가격·시장·품질 숫자는 Claim과 연결해야 한다.

관계에는 의존 당시 대상의 target_fingerprint를 저장한다. 원문 JSON의 객체는 locator={document_id, json_pointer}와 실제 원문 항목의 payload_sha256로 고정한다. OpenAPI의 정규화한 요약과 실제 operation 원문의 해시를 혼동하지 않는다. validate는 pointer 존재와 원문 항목의 해시 일치도 확인한다.

전체 공통 필드와 관계 제약은 project_kit/registry/SCHEMA.md에 있다. 단순한 폴더 검색에 그치지 않고 실제 FR·SCR·AT·IT·MET·EXP·API·BUS·KMR 노드와 관계를 등록하는 것이 이 키트의 구체적인 온톨로지 구현이다.

세 가지 상태와 날짜를 분리한다

차원	저장 필드	해석
문서 편집 수명주기	status	draft, review, approved, superseded, archived, rejected
요구·결정의 채택	decision_state	proposed, accepted, rejected, not_applicable

차원	저장 필드	해석
검증의 종류	verification_level	unverified, source_documented, fixture_tested, product_tested, pilot_measured
시간	valid_from, valid_to, review_due	효력 구간과 재검토 기한

승인 문서는 현재 작업의 편집 기준선일 수 있지만 제품은 미구현일 수 있다. 공급자 공식 문서를 확인한 source_documented는 모델 품질 실측이 아니다. 계산기의 fixture 통과는 실제 채택 영상 1초의 원가 검증이 아니다. 문서의 승인 범위를 editorial_baseline으로 기록하고, 승격 명령은 decision_state와 verification_level을 바꾸지 않는다.

Claim/Assumption의 claim_type은 fact, assumption, target, inference로 구분한다. 가정이나 목표를 실측 결과 상태로 직접 바꾸려 하면 오류를 낸다. 실제 결과가 생기면 새 Claim을 만들고 해당 Test/Experiment와 증거 Document를 연결한다. 검증 레벨이 product_tested 또는 pilot_measured라면 그 범위의 실행 기록과 증거 문서가 필요하다. 이 선언 검사는 증거 파일 자체가 진실인지 독립 감사하는 기능을 대신하지 않는다.

날짜도 구분한다. 원문을 가져온 날은 출처를 새로 확인한 날이 아니다. Source의 원래 확인일을 보존하고 확인일이 없으면 SOURCE_CHECK_UNKNOWN으로 표시한다. Refero 등 캡처 기반 자료는 원래 관측 단계와 checked_at_basis를 남겨 캡처일을 라이브 사이트 작업 실행일로 해석하지 않게 한다. 초기 import의 기본 검토 기한은 원래 확인일부터 30일이며, 가격·모델 기능처럼 변동성이 큰 자료는 더 짧은 검토 주기를 사람이 지정한다.

CLI 기능과 실제 동작

Python 3.11 이상을 사용한다. 별도 pip install, API 키, DB 연결은 필요하지 않다. 표준 argparse로 하위 명령을 읽고, json으로 sidecar를 처리하며, hashlib의 SHA-256으로 파일을 고정한다. 공식 Python 문서에 각 기능이 정의되어 있다. [S-P01, S-P02, S-P04]

명령	핵심 입력	결과와 실패 조건
validate	registry, 선택한 기준일	해시·관계·시점·순환 검사; 오류 시 실패
status, freshness	선택한 기준일	현재 head, 상태별 수, 검토·의존성 경고
search	검색어, task, 명시적 포함 옵션	현재 승인 기록 중심의 검색 결과와 상태
impact ID	객체 ID 또는 안정 키	전이적 영향 객체와 경로
context	task, 출력 경로, 예산, 선택 ID	제한된 Markdown 문맥과 receipt

명령	핵심 입력	결과와 실패 조건
context-check	기존 문맥/receipt 경로	현재 registry·파일·head와 일치 여부
ingest	파일/폴더, namespace	Document 등록 제안
import-catalogs	지원 카탈로그 폴더	실제 구조화 객체와 관계 등록 제안
merge	objects 배열 sidecar	기존 ID를 덮어쓰지 않는 추가
promote, archive	대상·이유·actor·revision	명시적 상태 변경과 감사 기록

읽기 명령은 원문과 registry를 바꾸지 않는다. 등록·승격·보관 명령도 기본은 dry run이다. --apply, --expected-revision, --actor를 명시해야 쓰기가 실행된다. 같은 로컬 registry에는 잠금 파일로 writer를 하나만 허용하고, 다른 작업자가 먼저 변경했다면 예상 revision 불일치로 중지한다.

쓰기 시 같은 디렉터리의 임시 파일에 JSON을 저장한 뒤 교체한다. 감사 이벤트를 같은 registry 안에 넣어 객체 변경과 감사 기록이 별도 파일에서 엇갈리지 않게 했다. Python os 문서의 O_EXCL과 replace를 이용한 로컬 구현이다. 모든 OS·공유 네트워크 드라이브의 분산 트랜잭션이나 장애 복구 보장으로 확대 해석하지 않는다. [S-P03]

자료의 권한은 로컬 파일과 Git 운영 정책에 의존한다. actor 문자열은 인증된 전자서명이 아니다. 별도 결재 서버나 조직 권한 시스템을 구현했다고 주장하지 않는다.

문맥 생성 계약

다음 명령은 승인 기준선에서 업무별 문맥을 만든다.

```
python kb.py context --task business --out generated/context/business-001.md
python kb.py context --task product --out generated/context/product-001.md
python kb.py context --task pitch --out generated/context/pitch-001.md
python kb.py context-check generated/context/pitch-001.md
```

지원 task는 business, product, pitch, technical, validation, operations다. --id를 반복하면 특정 요구·화면·주장에 집중할 수 있다. 기본은 현재 승인 head 중 유효하고 stale이 아닌 자료다. 초안·과거·stale 자료가 필요하면 별도 포함 옵션을 명시하고 상태 라벨을 유지한다.

기본 예산은 **추정 6,000토큰**, 객체별 원문 조각은 **최대 1,200자**다. 토큰 추정기는 ceil(UTF-8 바이트/3)이며 실제 모델 토크나이저가 아니다. 실제 토큰 상한을 보장하지 않고 예산 추정치·선택 수·제외 수를 표시한다. 텍스트 전체를 통째로 넣는 방식으로 400KB 문서를 문맥에 밀어 넣지 않는다.

문맥에는 ID·버전·출처·확인일·효력·검토 기한·해시·검증 수준·의존 관계를 넣고, 실제 원문 일부만 가져온다. Claim에 연결된 직접 근거는 Source citation ID와 URL·확인 범위까지 함께 표시한다. 영업 수치나 품질 목표를 사용할 때 ASSUMPTION_NOT_OBSERVED_RESULT, TARGET_NOT_OBSERVED_RESULT, UNVERIFIED_COST, UNVERIFIED_QUALITY 같은 라벨이 유지된다.

선택 자료와 그 안의 지시문은 untrusted data라고 명시한다. CLI는 문서에 적힌 명령을 실행하거나 네트워크로 보내지 않는다. 이 표시는 모델 자체의 보안 격리 기능을 보장하는 것은 아니므로 실제 에이전트도 사용자 지시와 자료 내용을 구분해야 한다.

receipt에는 문맥 파일 해시, 사용한 registry 해시·revision, 선택 ID와 의미 지문, 당시 head, 제외 이유를 저장한다. 다음 작업을 반영하기 전 context-check가 문맥·자료·head 변경을 검사한다. 무관한 registry 변경이라도 보수적으로 재생성을 요구한다. 동일 출력 이름을 덮어쓰지 않고 새 작업 이름을 사용한다.

문맥이 생성된다는 사실만으로 사업계획·제품·PPT가 완료되지는 않는다. 각 산출물은 작성, 주장별 근거 연결, 검토, 새 문서 등록과 승격을 통해 기준선으로 확정한다.

최신성 영향 전파와 오래된 PPT

변경 감지는 네 가지를 구분한다. 파일 SHA-256이 달라지면 원본 무결성 오류다. 기존 객체의 의미 지문이 바뀌면 의존 항목 변경 경고다. 현재 head가 다른 버전으로 이동하면 대체된 근거 경고다. 출처 재검토 기한이 지나면 최신성 경고다. 이러한 문제는 선언된 관계를 따라 하위 소비자에 전파한다.

예를 들어 가격 가정이 바뀌었는데 사업계획과 PPT가 이전 Claim을 계속 참조하면 둘을 stale로 표시한다. 사용자는 새 근거를 승인하고, 비용을 재계산하며, 사업 문장과 슬라이드를 수정한 뒤 새 Document 버전을 등록한다. CLI는 기존 PPT의 숫자를 직접 바꾸거나 이미 공유한 파일을 회수하지 않는다.

역사 자료가 과거 근거를 참조하는 것은 정상이다. archived/superseded 객체에는 현재성 경고를 불필요하게 누적하지 않는다. 다만 과거 파일도 해시, 존재 여부, 관계 ID, JSON pointer와 순환 검사를 계속 받는다. 현재 문서가 대체된 근거를 쓰는 경우에는 경고를 유지한다.

현재 구현은 연결된 컨테이너 파일의 변경을 보수적으로 처리한다. JSON 카탈로그 한 항목을 고치면 해당 컨테이너의 해시가 바뀌므로 여러 객체가 stale이 될 수 있다. 이는 잘못된 최신성 통과보다 재검토 범위를 넓게 잡는 선택이다. 항목별 의미 diff로 경고 범위를 자동 축소하는 기능은 아직 없다.

업무별 사용자 흐름

사업계획 작성자

현재 사업 문맥과 원가 모델 버전을 읽는다. 공급자 단가, 후보 배수, 채택률, 환율, 월 제작량, 지원 시간은 각각 Source 또는 Assumption에 연결한다. 모델링 결과는 실측 결과로 부르지 않는다. 가격이나 가정을 바꾸면 impact로 사업계획·비교표·PPT의 영향을 확인하고 새 버전을 만든다. BUS 요구와 EXP-C 비교 계획을 통해 실제로 확인해야 할 항목을 남긴다.

제품 개발 담당자

FR 또는 KMR로 검색하고 관련 Screen·API·Test·Metric을 확인한다. FR의 문서 정의, fixture 준비, 실행 결합, 구현 상태, 실제 통과를 구분한다. 범위 변경을 새 Decision과 Requirement 버전으로 남기고 API·화면·시험이 새 요구를 참조하도록 수정한다. 프로젝트 CLI의 요구를 제품 런타임 FR에 끼워 넣지 않는다.

발표자료 작성자

pitch 문맥과 receipt를 받아 메시지·근거·가정을 분리한다. “목표 90%”를 “달성 90%”로 바꾸지 않는다. 문서 Source의 관측 범위를 각 주장에 맞게 인용한다. 최종 PPT를 Document로 등록할 때 주요 비용·품질·시장 Claim과 사업 결정을 연결한다. 이전 발표자료의 영향 경고가 사라졌는지 검토하고 새 버전을 승격한다.

통합 담당자 또는 프로젝트 소유자

모델별 작업 범위와 기준 revision을 정하고 sidecar patch를 모은다. 각 변경의 dry run을 검토한 뒤 현재 revision에서 적용한다. 새 head 승격은 편집 검토 범위로 기록한다. 실제 요구 채택과 실험 결과는 별도 기록으로 보존한다. 오래된 파일은 대체·보관 상태와 보존 정책을 구분한다.

여러 모델의 동시 편집과 복원

모델마다 다른 초안 파일이나 Git worktree를 사용하는 방식을 권장한다. Git worktree는 한 저장소의 여러 분기를 동시에 작업하는 기능을 제공한다. 의미 충돌을 자동 해결하는 기능은 아니다. [S-P05]

각 작업자는 시작 commit, registry revision, 문맥 receipt를 남긴다. 동일한 JSON을 여러 모델이 직접 덮어쓰게 하지 않고 추가 patch를 제안한다. 통합 담당자는 최신 기준에서 patch를 다시 적용한다. 동일 로컬 경로의 충돌은 잠금·revision 검사가 막고, 서로 다른 worktree에서 생긴 충돌은 Git 검토와 감사 번호 검사로 다룬다. 양쪽 감사 기록을 단순히 이어 붙이면 번호 충돌이 검출된다.

잠금 파일은 오래됐다는 이유로 자동 삭제하지 않는다. 기록된 PID와 실제 작업 종료 여부를 확인하고 중단 결과를 검토한 뒤 운영자가 그 잠금 파일 하나만 처리한다. 원문·archive를 넓은 패턴으로 삭제하는 기능은 제공하지 않는다.

복원은 과거의 파일 바이트를 새 버전으로 복사하고 복원 이유·원래 commit을 기록하는 방식으로 진행한다. 과거 문서가 사용한 근거도 현재 유효한지 다시 검토한다. registry가 손상되면 사본을 보존하고 마지막 정상 Git 상태의 registry와 원문을 함께 복원한 뒤 validate를 실행한다. 레지스트리만 복원해서 파일 해시와 어긋난 상태를 정상이라고 보지 않는다.

실제 시험 결과와 범위

project_kit/tests/test_kb.py의 **16개 로컬 fixture 시험을 실제 실행해 통과**했다. CLI_TEST_REPORT.json에 실행 환경, 코드·시험 파일 해시, 시험 목록, 범위를 기록했다. Python unittest는 fixture·test case·runner를 제공하며 이번 실행도 이 범위의 로컬 동작 검사다. [S-P06]

시험 묶음	확인한 실패와 정상 동작	범위
최신 기준·보관	mtime가 새로워도 draft를 head로 선택하지 않음	CLI
무결성·영향	파일 변경, 대체된 비용 근거가 사업계획·PPT에 전파	CLI fixture
연결·순환	깨진 ID·pointer, 의존·대체 순환을 실패 처리	CLI
근거 상태	비용·품질 가정 라벨, fixture를 제품 증거로 오인하지 않음	CLI
문맥	예산 제한, receipt, 기존 문맥의 변경 감지	CLI
동시 수정	두 writer의 같은 revision 덮어쓰기 방지·감사 충돌	로컬 파일
실제 형식 import	카탈로그·OpenAPI 원문 pointer/hash와 Source 필드 변형	CLI
경로·읽기	경로 탈출·원문 덮어쓰기 차단, 읽기 명령의 무변경	CLI

별도로 실제 v2 카탈로그와 v1 archive의 임시 통합을 실행했다. 해당 확인 시점의 입력에서 current 835개, archive 722개, 총 1,557개 객체를 읽었고 오류·경고는 0이었다. 이 수는 그 입력 스냅샷의 import 확인 결과이며, 최종 패키지에 문서나 출처가 추가되면 객체 수는 달라진다. 최신 최종 수는 배포된 registry의 status와 최종 검증 기록을 따른다. 기존 v1 문서는 수정하지 않았다.

제품 생성, 모델 공급자, 결제, 실제 플랫폼 접근 제어, 고객 선호 재현, 시장 수요, 실제 제작비, 실제 PPT 렌더는 이 시험에 포함되지 않았다. .pptx 이름을 가진 시험 파일은 영향 전파용 바이트 fixture다. 실제 제품 시험이나 발표자료 생성 시험을 통과한 것으로 보고하지 않는다.

KMR-001~020 전체의 조직 운영 정착, 실제 사람의 검토 품질, 장기 백업 복원, 동시 분기 통합 습관까지 자동 시험으로 완료했다고 보지 않는다. 도구가 제공하는 검사와 프로젝트 운영자가 수행할 통제를 별도로 확인한다.

운영 수용 기준과 이후 확장

초기 운영의 수용 기준은 문서 수를 줄이는 것이 아니라 잘못된 기준 사용을 발견하는 것이다. 현재 head가 하나로 정해져 있고, 각 중요 주장에 Source 또는 가정·실험 상태가 붙으며, 가격·범위 변경이 소비 문서에 전파되고, 오래된 문맥을 재사용할 때 경고가 나와야 한다. 모델과 사람은 수정 이유와 새 버전을 남겨 다음 작업자가 판단 근거를 확인할 수 있어야 한다.

검색 품질이나 처리량이 실제로 부족해지면 색인·DB·그래프 저장을 검토한다. 그때도 원문과 승인 manifest를 원본으로 유지하고 재생성 가능한 검색 계층을 분리한다. 초기부터 Neo4j나 벡터 DB를 필수로 두지 않는다. 문서의 문장을 모델이 잘 이해하게 만드는 일과 승인·버전·근거를 정확히 관리하는 일은 서로 다른 검증 과제다.

자동 웹 확인, 슬라이드 내부 의존 추출, 모델 토크나이저 기반 예산, 인증된 결재, 협업 UI, 자동 작업 배정, 장기 백업 서버는 이후 범위다. 구현할 경우에도 외부 게시·유료 호출·삭제 권한은 자료 속 지시문이나 문서 승인 상태에서 자동으로 파생하지 않는다.

Windows PowerShell과 Unix 실행, 최초 등록, Source/Claim sidecar 작성, 승격·대체·보관·복원과 여러 모델 통합의 상세 명령은 project_kit/docs/OPERATIONS.md에 있다. 프로젝트 에이전트의 작업 규칙은 project_kit/AGENTS.md를 따른다.

기술 근거

- [S-P01] Python argparse: https://docs.python.org/3.11/library/argparse.html
- [S-P02] Python hashlib: https://docs.python.org/3.11/library/hashlib.html
- [S-P03] Python os: https://docs.python.org/3.11/library/os.html
- [S-P04] Python json: https://docs.python.org/3.11/library/json.html
- [S-P05] Git worktree: https://git-scm.com/docs/git-worktree
- [S-P06] Python unittest: https://docs.python.org/3.11/library/unittest.html

확인일은 2026-10-23이다. 이 공식 자료들은 사용한 표준 기능의 근거이며 신규 플랫폼의 성능·품질·사업성 검증 자료가 아니다. 출처 레지스트리는 research/project_ontology_sources.json에 있다.